

This is a solid package manager implementation with good architectural patterns. The hybrid SQLite + text manifest approach is sensible, and the natural-language CLI is user-friendly. I'll address your specific concerns, then provide general observations.

1. SQL Injection

Status: Excellent — all queries are properly parameterized

Throughout `database.py`, every query uses parameterized statements (`%` placeholders with tuples). Examples:

```
self.conn.execute("SELECT * FROM installed WHERE name = ?", (name,))
self.conn.execute("INSERT OR IGNORE INTO depends ... VALUES (?, ?, ?)", (package_id,
dep_name, dep_type))
```

No vulnerable patterns found. There's no string concatenation or f-string formatting of user input into SQL queries.

One minor concern: The search method uses `LIKE ?` with `f"%{term}%"` — this is fine because `term` is still bound as a parameter, not interpolated. The `%` characters are part of the parameter value, not the SQL string.

2. File Operation Safety

Status: Several vulnerabilities, particularly symlink attacks and TOCTOU races

Symlink Attacks During Extraction

The installer extracts archives directly to the target filesystem with `tar -xzf`:

```
result = subprocess.run(
    ["tar", "-xzf", str(archive_path), "-C", str(self.root),
    "--no-overwrite-dir", "--keep-directory-symlink"],
    capture_output=True, text=True
)
```

Problem: tar with these flags does **not** protect against symlink attacks where an archive contains a symlink pointing outside the extraction root, and later files are written through that symlink.

Exploit example: An archive containing:

```
etc/passwd -> /etc/passwd      # symlink
etc/passwd/anything           # writes to /etc/passwd as a directory!
```

While `--keep-directory-symlink` helps, the classic "symlink traversal" attack where a symlink is created then a file is written through it is still possible.

Fix: Use `tar --absolute-names --overwrite`? Better yet, extract to staging and use `os.path.realpath()` checks:

```
def safe_extract_to_root(staging_dir, root):
    for item in staging_dir.rglob("*"):
        rel = item.relative_to(staging_dir)
        dest = root / rel
        # Resolve symlinks and check that dest is within root
        try:
            real_dest = dest.resolve()
            if not str(real_dest).startswith(str(root.resolve())):
                raise ValueError(f"Path traversal attempt: {rel}")
        except (OSError, ValueError):
            # Broken symlink or traversal
            continue
        # Copy/move the file
```

TOCTOU Races in File Removal

In `remover.py`:

```
if os.path.isfile(abs_path):
    config = self.db.conn.execute(...).fetchone()
    if config and config[0]:
        current = _sha256(abs_path)
        if current != config[0]:
```

```
    preserved_configs.append(f["path"])
    continue
```

```
# Later...
os.remove(abs_path)
```

Between checking the file exists and removing it, an attacker could replace the file with a symlink pointing to /etc/shadow. The removal would then delete the target.

Fix: Use `os.lstat()` and `os.unlink()` directly, or open the file descriptor before checking:

```
try:
    # Use lstat to avoid following symlinks
    st = os.lstat(abs_path)
    if stat.S_ISLNK(st.st_mode):
        # Don't remove symlinks that might point outside
        continue
    # Then unlink
    os.unlink(abs_path)
except (OSError, PermissionError):
    pass
```

Path Traversal in Archive Member Names

You extract to staging first (good), but the staging extraction itself doesn't validate paths:

```
subprocess.run(["tar", "-xzf", str(archive_path), "-C", str(staging)], ...)
```

If an archive contains `../../../../etc/cron.d/evil`, tar will happily extract it outside staging. While staging is a temp directory, this could still overwrite system files if the archive is malicious.

Fix: Before extraction, list archive contents and validate:

```
def safe_extract_archive(archive_path, staging_dir):
    # List contents
    result = subprocess.run(
        ["tar", "-tzf", str(archive_path)],
```

```

        capture_output=True, text=True
    )
    for member in result.stdout.splitlines():
        # Normalize and check for path traversal
        parts = Path(member).parts
        if '..' in parts or member.startswith('/'):
            raise ValueError(f"Dangerous archive member: {member}")

    # Then extract
    subprocess.run(["tar", "-xzf", str(archive_path), "-C", str(staging_dir)], ...)

```

Race in Staging Cleanup

```

staging = Path(tempfile.mkdtemp(prefix=f"pkm-{name}-"))
try:
    # ... extraction and deployment
finally:
    shutil.rmtree(staging, ignore_errors=True)

```

mkdtemp is secure (random suffix), and shutil.rmtree is fine, but there's a small window between the try and the actual extraction where the directory could be replaced. Acceptable for a package manager.

Missing --no-same-owner for tar

When extracting as root, tar preserves file ownership from the archive. If an archive contains files owned by uid 0 (root), that's fine. But if it contains files owned by uid 1000 that doesn't exist on the system, extraction will fail or set weird owners. Consider --no-same-owner to force root ownership.

3. Config File Preservation

Status: Generally correct, but has a logic flaw

The preservation logic in remover.py:

```

if f["path"].startswith("/etc/"):
    if os.path.isfile(abs_path):
        config = self.db.conn.execute(
            "SELECT original_checksum FROM config_files WHERE path = ?",
            (f["path"],)
        ).fetchone()
    if config and config[0]:
        try:
            current = _sha256(abs_path)
            if current != config[0]:
                preserved_configs.append(f["path"])
                continue # Skip removal
        except (OSError, PermissionError):
            pass

```

Problems identified:

1. **Missing directory case:** If a user modifies a config file that's a symlink (e.g., `/etc/resolv.conf → /run/systemd/resolve/stub-resolv.conf`), `_sha256()` will follow the symlink and checksum the target. The original checksum in the database might be for the symlink itself. This could cause false positives/negatives.
2. **Config files added after install:** If a package creates a config file on first run (e.g., ssh creating host keys), those files won't be in the database at all. They won't be removed (good), but they also won't be preserved if they should be.
3. **No protection for --force removal:** The force flag bypasses dependency checking but still preserves configs. This is probably correct — force should remove the package but not nuke user data.
4. **The continue is correct** — skipping removal but still logging as preserved.
5. **Missing restoration of original config:** If the user wants to reinstall, the preserved config file will remain, which might be what you want (user's custom config survives reinstall). This is a design choice, not a bug.

Edge case bug: When a config file is preserved, the file isn't removed, but the directory cleanup loop later might try to remove the parent directory if empty. Since the config file still exists, `os.listdir()` on the parent will find it, so the directory won't be removed. Correct.

4. Dependency Resolution

Status: Works, but limited to installed packages

The removal dependency check:

```
rdeps = self.db.get_reverse_depends(name)
if rdeps:
    dep_list = ", ".join(f"{d['name']}" for d in rdeps)
    return False, f"Cannot remove {name}: {len(rdeps)} package(s) depend on it..."
```

This correctly prevents removal of packages that others depend on.

Limitations to note:

1. **No version constraints:** Dependencies are tracked by name only. If Package A depends on openssl $\geq$ 3.0 and B depends on openssl $\geq$ 1.1, the system can't detect conflicts. For a from-scratch distribution where you control all packages, this is acceptable.
2. **No optional dependencies:** dep_type exists (build, host, runtime) but removal doesn't distinguish — all types block removal. This is safe but over-constraining. A build dependency shouldn't block removal at runtime.
3. **No dependency resolution on install:** The installer doesn't check that dependencies are satisfied before installing. It assumes the build system already ensured the dependency graph is satisfied. This is fine for your use case but worth documenting.
4. **No circular dependency detection:** The get_reverse_depends query could theoretically return a package that depends on itself (circular), but the database UNIQUE constraint prevents that.

Suggested improvement for removal:

```
# Only runtime deps should block removal; build/host deps are compile-time only
runtime_rdeps = [d for d in rdeps if d['type'] == 'runtime']
if not force and runtime_rdeps:
    return False, f"Cannot remove {name}: {len(runtime_rdeps)} runtime dependents..."
```

5. Archive Extraction Security

Status: Multiple issues — Tar slip, path traversal, symlink attacks

Tar Slip Vulnerability

The classic "tar slip" occurs when an archive contains a symlink to a directory outside the extraction root, followed by files that write through that symlink. Your staging extraction is vulnerable:

```
# Extracts to staging without validation
subprocess.run(["tar", "-xzf", str(archive_path), "-C", str(staging)], ...)
```

Exploit: Archive contains:

```
tmp/ -> /tmp          # symlink to /tmp
tmp/evil                # writes to /tmp/evil (outside staging!)
```

Mitigation: Use tar --absolute-names and validate after extraction:

```
def validate_extraction(staging_dir, root="/"):
    staging_dir = Path(staging_dir).resolve()
    for item in staging_dir.rglob("*"):
        try:
            # Check if the resolved path is still under staging_dir
            resolved = item.resolve()
            if not str(resolved).startswith(str(staging_dir)):
                raise ValueError(f"Path traversal: {item.relative_to(staging_dir)} -> {resolved}")
        except (OSError, ValueError):
            # Broken symlink or traversal attempt
            raise
```

Missing Archive Integrity Verification

You verify the archive's existence but not its checksum. The build system creates archives with checksums in the manifest, but the installer doesn't verify:

```
# Should verify archive SHA256 before extraction
expected_checksum = ... # From database or manifest
if not self.verify_archive(archive_path, expected_checksum):
    return False, "Archive checksum mismatch"
```

Fix: Store archive checksums in the database when registering packages, and verify on install.

Archive Source Trust

You search ARCHIVE_DIR for any file matching the package name. An attacker who can write to /var/lib/igos/archives/ could replace a legitimate archive with a malicious one. Consider signing archives or at least verifying checksums against a trusted source.

6. General Code Quality

Strengths:

- Clean separation of concerns (database, installer, remover, verifier, cli)
- Good use of SQLite with foreign keys and WAL mode
- Proper error handling in most places
- History logging for auditability
- Config file preservation is thoughtful

Issues and improvements:

Database Connection Management

The __main__.py has:

```
db = PackageDB(args.db)
try:
    # commands...
finally:
    db.close()
```


This is good, but if an exception occurs during command execution, the finally block still runs. However, PackageDB has no `__enter__`/`__exit__` to make it a context manager. Add:

```
def __enter__(self):
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    self.close()
```

Then use:

```
with PackageDB(args.db) as db:
    # commands...
```

Missing Rollback on Install Failure

If installation fails after partial file deployment, there's no rollback. The database is updated only after successful deployment, but files already written to disk remain. Consider:

1. Extract to staging
2. Validate staging contents
3. Take filesystem snapshot or use a temporary overlay
4. Deploy to target
5. If any step fails, restore from backup or at least log the partial state

Race Condition in `_find_archive`

```
for f in sorted(ARCHIVE_DIR.iterdir(), reverse=True):
    if f.name.startswith(f"{name}-") and f.name.endswith(".igos.tar.gz"):
        return f
```

If multiple versions exist, this returns the lexicographically highest (due to `reverse=True`), not necessarily the newest. Use `f.stat().st_mtime` for proper version selection.

Hardcoded Paths

- `/var/lib/igos/packages` and `/var/lib/igos/archives` are hardcoded in multiple files
- The `root` parameter exists but isn't fully respected everywhere (e.g., `_write_manifest` writes to `self.root / "var" / ...` — good, but `MANIFEST_DIR` is a global constant)

Consider a configuration class that reads from `/etc/igos/pkg.conf` with sensible defaults.

`_parse_manifest` Fragility

```
elif ":" in line and not in_files and line.strip().startswith(meta.get("name", "\x00")):  
    # Description line: "bash: The GNU Bourne Again shell"  
    meta["description"] = line.split(":", 1)[1].strip()
```

This heuristic fails if the package name contains a colon or if the description spans multiple lines. The manifest format from the build system is more structured — consider using that structure rather than parsing.

Error Handling in `remove`

```
try:  
    if os.path.lexists(abs_path):  
        os.remove(abs_path)  
        removed_count += 1  
except (OSError, PermissionError) as e:  
    pass # Best effort — don't fail the whole removal
```

Silently ignoring all errors could hide real problems. At minimum, log the error to the history table or `stderr`. Consider which errors are fatal vs. ignorable.

Missing `--dry-run` Mode

For destructive operations (`remove`), a `--dry-run` flag would be valuable.

import_manifests Idempotency

The method checks if existing: continue, which is good. However, if a package was partially imported (files missing from DB but package exists), subsequent imports won't fix it. Consider a --force option to re-import.

SQLite Index Coverage

The schema has:

```
CREATE INDEX IF NOT EXISTS idx_files_path ON files(path);  
CREATE INDEX IF NOT EXISTS idx_files_package ON files(package_id);
```

Missing index on config_files.path (used in removal). Add:

```
CREATE INDEX IF NOT EXISTS idx_config_path ON config_files(path);
```

Also consider indexes on history.timestamp and history.package_name for faster history queries.

Thread Safety

The database is opened with default SQLite threading mode (serialized), which is fine for single-threaded use. But if you ever add parallel installs, you'll need connection pooling or a lock.

Summary Table

Area	Rating	Notes
SQL injection	10/10	Perfect — all queries parameterized
File operation safety	4/10	Tar slip, symlink attacks, TOCTOU races
Config preservation	7/10	Correct logic; symlink edge case

Dependency resolution	7/10	Works; no version constraints; build vs runtime
Archive extraction	3/10	Multiple vulnerabilities (tar slip, path traversal)
Code quality	7/10	Clean architecture; missing context manager; fragile parsing

Priority Fixes

1. **Tar slip vulnerability** — Validate archive members before extraction (critical)
2. **Symlink attack during extraction** — Use `--absolute-names` or validate resolved paths (critical)
3. **TOCTOU in file removal** — Use `lstat` + `unlink` without checking first (high)
4. **Archive integrity verification** — Store and verify checksums (high)
5. **Config file symlink handling** — Decide whether to follow or preserve symlinks (medium)
6. **Database context manager** — Add `__enter__`/`__exit__` (low)
7. **Remove silent error swallowing** — Log removal failures (medium)

Additional Recommendations

Add Pre-Install Validation

```
def pre_install_check(self, archive_path):
    # Verify archive integrity
    # Check for path traversal
    # Verify dependencies are installed
    # Check disk space
    # Check for conflicting files
```

Implement Package Groups

Consider adding "groups" (e.g., desktop, server) to allow installing related packages together.

Add --reinstall Command

Currently, users must remove then reinstall. A reinstall command would be convenient and could preserve config files automatically.

Signed Archives (Future)

For a production distribution, consider GPG-signing archives or using a manifest with signed checksums.

Overall, this is a well-architected package manager with a clear design. The security issues are fixable without major restructuring, and the core functionality (tracking, queries, removal) is solid. The hybrid SQLite + text manifest approach is particularly clever — you get query performance without losing human-readable auditability.